

AI TRAINING – DAY 6 ASSIGNMENT

Python Functions: Define → Pass → Return → Reuse

Learning Guidance + Refactoring Practice + Function-Based Utility Module

Level	Beginner → Medium
Suggested Time	3–4 hours
Core Topics	functions, parameters, return values, default arguments, *args, **kwargs
Daily Deliverable	Function-Based Utility Module
Main Focus	Turn repeated code into reusable functions

Important: What Is the Student Deliverable?

This assignment is your learning guidance. You do not need to submit a separate file for every activity. Your final Day 6 submission is ONE notebook containing your learning and testing, plus ONE Python utility module containing the final reusable functions.

```
Day_06_Function_Practice_YourName.ipynb
```

```
day_06_utility_module.py
```

Learning Goals

- Understand why functions are used in programming.
- Explain functions, parameters and return values in your own words.
- Create functions and call them correctly.
- Pass information using parameters.
- Return results using return.
- Use default arguments.
- Understand and use *args.
- Understand and use **kwargs.
- Identify common function errors and improve the solution.
- Refactor earlier loop programs into reusable functions.

How to Use This Assignment

1. Read and understand the concept before coding.
2. Write a small example before moving to the next concept.
3. Predict what the function will return before running it.
4. Try changing the arguments and observe the result.
5. Refactor repeated code instead of copying and pasting.
6. If you use AI for help, understand and explain the final solution yourself.
7. Keep your final reusable functions in the utility module.

PART A – Understand Functions

1. What is a Function?

Think about: Why put code inside a function instead of writing it every time?

```
def greet():  
    print("Hello!")  
  
greet()  
greet()
```

Explain: What is the function? What is the function call? How many times does it execute?

Ans:

Function is a block of code performs on a specific task. once the function is created we can use many it can leads to code reusability.

Function call : function call is nothing using or executing the already created function.

Output:

If executes two times because we call function two times.

2. Function with a Parameter

```
def greet(name):  
    print("Hello", name)  
  
greet("Ravi")  
greet("Anita")
```

Experiment by changing the values passed to name.

```
def greet(name):  
    print("Hello", name)  
  
greet("Deva")  
greet("Dini")
```

output:

Hello Deva

Hello Dini

3. Return Value

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
print(result)
```

Explain the difference between print and return.

Ans:

Print will display the information on display. and return will send the value back from the function to where it is required.

4. Multiple Parameters

```
def calculate_total(price, quantity):  
    return price * quantity  
  
print(calculate_total(100, 3))
```

Explain what each parameter represents.

Ans:

In the above program price refers to price of item. Quantity refers to the numbers of items.

Price=100

Quantity=3

PART B – Parameters vs Arguments

- Parameter: the variable written in the function definition.
- Argument: the actual value passed when calling the function.

```
def introduce(name, age):  
    return f"{name} is {age} years old"  
  
introduce("Rahul", 21)
```

Write three examples of your own and label parameters and arguments.

PART C – Default Arguments

```
def greet(name, city="Hyderabad"):  
    return f"{name} lives in {city}"  
  
print(greet("Ravi"))  
print(greet("Anita", "Bangalore"))
```

- What happens when city is not provided?
- What happens when city is provided?
- Why are default arguments useful?

Create at least 3 functions using default arguments.

Ex-1:

```
def add(a,b):    ## a,b are parameters.  
    return a+b  
print(add(2,3)) ## 2,3 are arguments.
```

Ex-2:

```
def info(name,age):    ## name,age are parameters.  
    return (f"my name is {name} . i am {age} years old.")  
print(info('deva',23)) ## deva,23 are arguments.
```

Ex-3:

```
def valid(marks): # marks are parameters  
    if marks >90:  
        return 'you are valid'  
    else:  
        return ' not valid'  
print(valid(int(input("enter your marks:")))) #input value is arguement
```

PART D – *args

Understand *args as a way to receive a variable number of positional arguments.

```
def add_all(*numbers):
    total = 0
    for number in numbers:
        total += number
    return total

print(add_all(10, 20))
print(add_all(10, 20, 30, 40))
```

Experiment with 0, 1, 2 and many arguments. Explain what *args gives you inside the function.

```
def arge_type(*args):
    return sum(args)
print(arge_type())
print(arge_type(10))
print(arge_type(10,20))
print(arge_type(10,20,30))
ans:
```

it gives us a tuple inside a function which contain all positional arguments.

PART E – **kwargs

Understand **kwargs as a way to receive a variable number of keyword arguments.

```
def show_profile(**details):
    for key, value in details.items():
        print(key, ":", value)

show_profile(name="Ravi", age=21, city="Hyderabad")
```

Add/remove profile details and explain what data you receive inside the function.

```
def kwargs_type(**kwargs):
    return kwargs
print(kwargs_type(name="deva"))
print(kwargs_type(name="deva", age=21))
add details:
```

```
def kwargs_type(**kwargs):
    return kwargs
print(kwargs_type(name="deva"))
print(kwargs_type(name="deva", age=21, stream="cyber"))
ans:
```

it receives data in the form of dictionary.

PART F – Combine the Concepts

```
def create_profile(name, city="Hyderabad", *skills, **details):
    print("Name:", name)
    print("City:", city)
```

```

    print("Skills:", skills)
    print("Other Details:", details)

    create_profile("Ravi", "Bangalore", "Python", "React", age=21, college="ABC Engineering College")

```

Modify this example and explain what each part receives.

Ans:

In the above code first print statement will take name from function call and print the name. In second print statement city will take value from the function call instead of default value. If did not give any value in function then value in the default will be taken. In third print statement it will take multiple values from the function call. Because we used *args. At final print statement it will take one or more key and values from the function call.

Output:

Name: Ravi

City: Bangalore

Skills: ('Python', 'React')

Other Details: {'age': 21, 'college': 'ABC Engineering College'}

PART G – Predict Before You Run

Predict the output first, then run each program.

```

def square(number):
    return number * number

x = square(5)
print(x)

```

Ans:

input: 5

function execution: square(5)

return value: 25

final output: 25

```

def greet(name="Student"):
    return "Hello " + name

print(greet())
print(greet("Phani"))

```

Ans:

1. Greet():

input: no argument

function execution: greet()

return value: "Hello Students"

final output: Hello student

2.greet("phani"):

input: "phani"

function execution: greet("phani")

return value: "Hello Phani"

final output: Hello phani

```
def total(*numbers):  
    return sum(numbers)
```

```
print(total(1, 2, 3))  
print(total(10, 20, 30, 40))
```

Ans:

1.total(1,2,3):

input: 1,2,3

function execution: total(1,2,3)

return value: 6

final output: 6

2.total(10,20,30,40)

Ans:

input: 10,20,30,40

function execution: total(10,20,30,40)

return value: 100

final output: 100

```
def profile(**data):  
    return data
```

```
result = profile(name="Ravi", age=21)
print(result["name"])
```

Ans:

input: name="Ravi", age=21

function execution: profile(name="Ravi", age=21)

return value: result = {
 "name": "Ravi",
 "age": 21
}

final output: ravi

For each: Inputs → Function execution → Return value → Final output.

PART H – Common Function Errors

For every bug: identify the problem, explain why it happens, fix it and test it.

Bug 1
def add(a, b)
 return a + b

What is missing?

Ans:

Calling the function is missed and colon after the define function.

```
def add(a, b):  
    return a + b  
print(add(2,3))
```

Bug 2
def add(a, b):
 print(a + b)

result = add(10, 20)
print(result)

Explain print vs return.

Ans: Print will display the information which is stored in result. We had added the values inside the print statement so it will print the value 30 and there is no return is give so by default it will give none.

Bug 3
def greet(name):
 return "Hello " + name

print(greet())

Why does this fail? Fix it using a default or supplied argument.

Ans: because positional argument is missing.

```
def greet(name):  
    return "Hello " + name  
  
print(greet('deva'))
```

```
Bug 4  
def calculate(price, quantity=1):  
    return price * quantity  
  
print(calculate())
```

Why does this still fail?

Ans: we given a value for one parameter .another argument is missing.

```
def calculate(price, quantity=1):  
    return price * quantity  
  
print(calculate(300))
```

```
Bug 5  
def add_all(*numbers):  
    return numbers + 10
```

What is numbers? Fix the function.

Ans: numbers is the parameter name and * give multiple parameters.

```
def add_all(*numbers):  
    return sum(numbers) + 10  
  
print(add_all(1, 2, 3, 4, 5))
```

output: 25

```
Bug 6  
def show(**details):  
    print(details)  
  
show("Ravi", 21)
```

Why is this not a correct use of **kwargs? Fix the call.

Ans: we are giving one values **kwargs requires both keys and values.

```
def show(**details):  
    print(details)  
  
show(name="Ravi", age=21)
```

PART I – Refactor Previous Loop Programs

Take your Day 5 loop programs and convert them into reusable functions. Refactor at least:

8. Multiplication table
9. Sum of numbers
10. Prime number checker
11. Palindrome checker
12. Find factors
13. Pattern printing

Example:

```
# Before
number = 7
for i in range(1, 11):
    print(number * i)

# After
def multiplication_table(number):
    for i in range(1, 11):
        print(number * i)

multiplication_table(7)
multiplication_table(12)
```

Do not merely put old code inside def. Ask: What should the function receive? What should it return? Can another program reuse it?

PART J – Function Practice Problems

Problem 1 – Temperature Converter

Create functions for Celsius → Fahrenheit and Fahrenheit → Celsius.

```
def temperature_converter(value, choice):
    match choice:
        case 1:
            return (value * 9/5) + 32
        case 2:
            return (value - 32) * 5/9
        case _:
            return "Invalid choice"
print("1. Celsius to Fahrenheit")
print("2. Fahrenheit to Celsius")
choice = int(input("Enter your choice: "))
value = float(input("Enter temperature: "))
result = temperature_converter(value, choice)
print("Result:", result)
```

Problem 2 – Even/Odd Function

Create a function that receives a number and returns whether it is even or odd.

```
def even_odd(vlaue):
    if value%2==0:
        return f"{value} is even number"
    elif value%2==1:
        return f"{value} is odd number"
    else:
        return "invalid number"
value=int(input("enter number:"))
result=even_odd(value)
print(result)
```

Problem 3 – Prime Function

Create reusable is_prime(number) returning True or False.

```
def is_prime(vlaue):
    count=0
    while value<=0:
        return "not a prime"
    for i in range(1,value+1):
        if value%i==0:
            count+=1
    return count==2
value=int(input("enter number:"))
result=is_prime(value)
print(result)
```

Problem 4 – Palindrome Function

Create reusable is_palindrome(value).

```
def is_palindrome(value):
    value = str(value)
    return value == value[::-1]
print(is_palindrome(input("enter input:")))
```

Problem 5 – Grade Function

Create a function that receives marks and returns a grade; handle invalid marks.

```
def get_grade(marks):
    if marks < 0 or marks > 100:
        return "Invalid marks"
    elif marks >= 90:
        return "A"
    elif marks >= 80:
        return "B"
    elif marks >= 70:
        return "C"
    elif marks >= 60:
```

```

        return "D"
    else:
        return "FAIL"
print(get_grade(int(input("enter your marks:"))))

```

Problem 6 – Shopping Total

Create a function receiving price and quantity and returning the total.

```

def calculate_total(price, quantity):
    return price * quantity
price=float(input("enter price:"))
quantity=int(input("enter quantity"))
print(calculate_total(price, quantity))

```

Problem 7 – Flexible Sum

Create add_all(*numbers) returning the sum of any number of values.

```

def add_all(*numbers):
    return sum(numbers)
print(add_all(*map(int, input("Enter numbers: ").split())))

```

Problem 8 – Profile Builder

Create build_profile(**details) returning a profile dictionary.

```

def build_profile(**details):
    return details
name = input("Enter your name: ")
age = input("Enter your age: ")
course = input("Enter your course: ")
profile = build_profile(name=name, age=age, course=course)
print(profile)

```

Problem 9 – Greeting Utility

Create a greeting function with a default city and optional name.

```

def greeting(name, city):
    return f"Hello {name} . you are from {city}"
name=input("enter your name:")
city=input("enter your city:")

print(greeting(name,city))

```

Problem 10 – Statistics Utility

Create functions for minimum, maximum and average of a collection of numbers.

```
def find_minimum(numbers):
    return min(numbers)
def find_maximum(numbers):
    return max(numbers)
def find_average(numbers):
    return sum(numbers)/len(numbers)
numbers = list(map(int,input("enter numbers:").split()))
print(find_minimum(numbers))
print(find_maximum(numbers))
print(find_average(numbers))
```

PART K – Main Project: Function-Based Utility Module

Create one reusable Python utility module, like a small library another Python program could import.

Your module must contain at least 10 reusable functions. Suggested functions:

- is_even(number)
- is_prime(number)
- is_palindrome(value)
- calculate_sum(numbers) or add_all(*numbers)
- calculate_average(numbers)
- find_factors(number)
- calculate_grade(marks)
- calculate_total(price, quantity=1)
- celsius_to_fahrenheit(celsius)
- fahrenheit_to_celsius(fahrenheit)

You may replace functions with your own useful utilities, but the module must demonstrate all Day 6 concepts.

Required Function Concepts in the Final Module

- At least 10 reusable functions.
- Functions with normal parameters.
- Functions that return values.
- At least 2 functions with default arguments.
- At least 1 function using *args.
- At least 1 function using **kwargs.
- At least 3 functions refactored from Day 5 loop programs.
- Input validation where appropriate.
- Clear and meaningful function names.

Example Module Usage

```
from day_06_utility_module import is_prime, calculate_grade

print(is_prime(17))
print(calculate_grade(82))
```

Your module should work when imported from another Python file.

PART L – Testing Your Functions

For important functions, test normal, boundary, invalid and different argument combinations where relevant.

- Normal input.
- Boundary input.
- Invalid input where applicable.
- Small and large values where relevant.
- Different combinations of arguments.

Create a small test section at the bottom of your notebook.

PART M – Explain Your Code

14. Why are functions useful?

Ans: functions are used for reusing the program for many times instead of writing every time. It leads to reusability.

15. What is the difference between a parameter and an argument?

Ans: parameters are the variables given with function and arguments are the values that are required by parameters.

16. What is the difference between print and return?

Ans: print displays the information on the display and the return is used to send a value back to where it is required.

17. Why use a default argument?

Ans: giving the values are automatically given to variables at the function definition. When there is no value is given

18. What does *args allow you to do?

Ans: when we want to give multiple values.

19. What does **kwargs allow you to do?

Ans: to give multiple key-value pairs.

20. Why is refactoring useful?

Ans: Refactoring is useful because it makes the code cleaner, simpler, and easier to maintain.

21. Which Day 5 program became easier to reuse after conversion?

Ans: the palindromic program.

22. Which function was hardest to design?

Ans: temperature converter because it values based on the user.

23. What bug did you face and how did you improve your solution?

Ans: I faced a problem at average function. I solved it by using a condition.

PART N – STUDENT DELIVERABLE – WHAT YOU MUST SUBMIT

Submit TWO files. One notebook for your learning/practice and one Python module for your final reusable functions.

```
Day_06_Function_Practice_YourName.ipynb
```

```
day_06_utility_module.py
```

The notebook must contain:

24. Your own explanations of functions, parameters and return values.
25. Default argument practice.
26. *args practice.
27. **kwargs practice.
28. Output-prediction exercises.

29. Debugging exercises and corrected solutions.
30. Solutions to the 10 function practice problems.
31. Refactoring notes for Day 5 programs.
32. Testing of your utility functions.
33. Final reflection.

The Python utility module must contain:

34. At least 10 reusable functions.
35. Normal parameters and return values.
36. At least 2 default-argument functions.
37. At least 1 *args function.
38. At least 1 **kwargs function.
39. At least 3 functions refactored from Day 5 programs.
40. Meaningful names and readable code.

You DO NOT Need to Submit

- A separate file for every function.
- A separate file for every practice problem.
- Screenshots of every output.
- A separate Word/PDF document.

Submission Checklist

- ☐ Functions explained in my own words.
- ☐ Parameters and arguments practiced.
- ☐ Return values practiced.
- ☐ Default arguments practiced.
- ☐ *args practiced.
- ☐ **kwargs practiced.
- ☐ Debugging completed.
- ☐ 10 function problems completed.
- ☐ Previous loop programs refactored.
- ☐ Utility module contains at least 10 functions.
- ☐ Utility module contains default args.
- ☐ Utility module contains *args.
- ☐ Utility module contains **kwargs.
- ☐ Functions tested.
- ☐ Both files are ready to submit.

Suggested Evaluation

Area	Weight
Concept understanding + explanation	15%
10 Function Problems	25%
Refactoring Day 5 Programs	20%
Utility Module	25%
Testing + Debugging	10%
Code Quality + Reflection	5%

DAY 6 PRINCIPLE

Don't write the same logic again and again. Write it once, make it reusable, and call it whenever you need it.